Embedded Data Structure
(Lecture Note 5)

Revisit C Programming (4)

2025. 3. 18

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of the Last Class

- **Basic concept of advanced operations for pointers**
  - **Pointer and string**
    - Inputs of functions related to string are pointers in general
    - Remember the relation between pointer of pointer and string
  - **Pointer and its applications**
    - Array of pointers, function pointer, and array of function pointers
  - **Remember how to read/write a file**

Memory Allocation (1/4)

KU KONKUK UNIVERSITY

▪ Dynamic management with pointers

- **Two basic types of memory allocation**

  - int *p = (int *)**malloc**(height*width*sizeof(int));
  - int *p = (int *)**calloc**(height*width, sizeof(int));

  Consider the return type

  ※ malloc → initialize space with garbage values, calloc → 0

  *** Free the memory if you want to …**

```
int *p = (int *)calloc(height*width, sizeof(int));

for(y = 0; y < height*width; y++){
    p[y] = 9;
}

free(p);
```

← Free the "address" (i.e., another pointer can reserve it !)

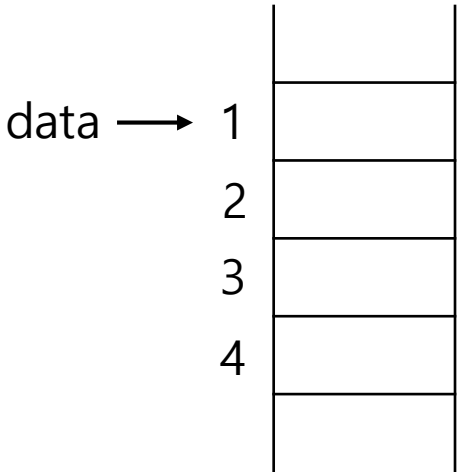
Memory Allocation (2/4)

- **Dynamic management with pointers – cont'd**
  - **Interpret the allocation procedure with drawing**
    - For example, consider a specific data whose size 9x9

```
float *data = (float *)calloc(9*9, sizeof(float));

// load data using "data"
// compute the average
for(y = 0; y<9*9; y++){
   avg += data[y];
}
avg /= (9*9);

free(data);
```



Where is our data stored ?

※ If we do not free the memory space, what happens ? (draw it !)

Memory Allocation (3/4)

KU KONKUK UNIVERSITY

▪ Allocation with pointer of pointers

- **If you want to reserve the memory with pointer of pointers …**

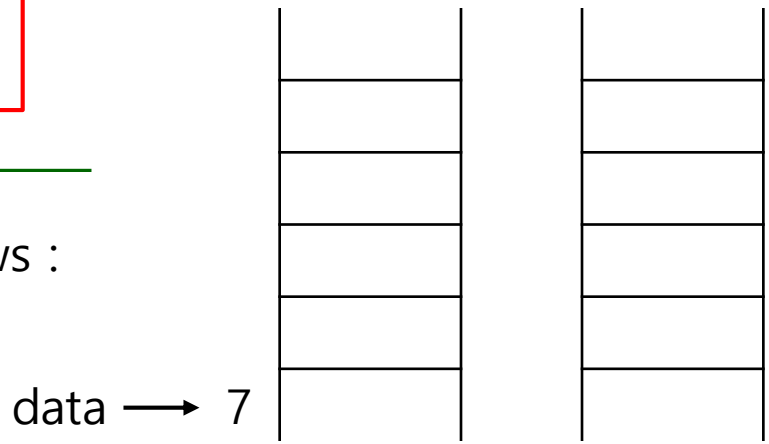
  - Try to conduct memory allocation by using "float **data"

    (For example, 9x39 data is stored in a two-dimensional array)

float **data = [ ]

Hint : use the for loop

※ Draw the overall procedure as follows :



Memory Allocation (4/4)

- **Simple implementation example**
  - **Compute the average of your own data (input by keyboard)**

```
#include <stdio.h>
#include <stdlib.h>

void main(){
    int len;
    printf("Input your square matrix size :");
    scanf("%d", &len);

    // memory allocation using "int **matrix"

    // compute the average

}
```

Structure (1/5)

- **Structure including a set of different types of data**
  - **"struct" instruction**

```
struct Tag{
Data_type        member 1;
Data_type        member 2;
...
};
```

※ What is the difference btw. array and structure ?

```
struct ProgramClass{
    char name[100];
    int ID;
    int score;
    float attendance;
    float grade;
};
```

Tag (it is not a variable !)

Structure member

```
struct ComplexNum{
    float re;
    float im;
};
```

Example of structures

Initialization

- **Syntax and initialization scheme**

```
#include <stdio.h>

void main()
{
    struct ProgramClass s1;
    struct ProgramClass s2, s3;

    s1 = {"Kim", 1, 90, 1, 4.3};
}
```

K	i	m	₩0	…	1	90	4.3

In your memory space …

Difference with "*name" ?

```
struct ProgramClass{
    char name[100];
    int ID;
    int score;
}s1;
```

```
struct ProgramClass{
    char name[100];
    int ID;
    int score;
}s1 = {"Kim", 1, 90};
```

[Other initialization strategies]

Structure (3/5)

▪ Referencing

- **We can access the structure member using "." as follows:**

```
#include <stdio.h>
#include <stdlib.h>

void main()
{
    struct ProgramClass s1;
    struct ProgramClass s2, s3;

    strcpy(s1.name, "Kim");
    s1.ID = 1;
    s1.score = 90;
    s1.attendance = 1.0;
    s1.grade = 4.3;
}
```

```
#include <stdio.h>

struct ComplexNum{
    float re;
    float im;
};

void main()
{
    Make your code !
}
```

KU KONKUK UNIVERSITY

Structure with typedef

- **We can redefine data type (or structure) efficiently**

: For example, **typedef** unsigned char BYTE

: For example, **typedef** struct student STUDENT

```
#include <stdio.h>

typedef unsigned char BYTE;

void main()
{
   BYTE val;
}
```

```
Typedef struct student STUDENT;

STUDENT s;
```

```
typedef struct ProgramClass{
   char name[100];
   int ID;
   int score;
   float attendance;
   float grade;
}ProgramClass;
```

➡ Is it Ok ?

Then, main advantage of typedef is …

Structure (5/5)

▪ Practical example

- **Complex multiplication using typedef and structure**

```
#include <stdio.h>

Typedef struct _complex{
    float re;
    float im;
}CNum;

CNum multiply(CNum a, CNum b)
{
    CNum c;

    [redacted]
    [redacted]
}
```

```
void main()
{
    CNum a = {1.3, 3.9};
    CNum b = {-3.3, 8.4};
    CNum result;

    [redacted]

}
```

➡ **Check the result and print out**

Summary

- **Memory allocation and structure**
  - **Memory allocation : key technique in embedded systems**
    - Remember the usage of "malloc" and "calloc"
    - Remember the memory allocation based on pointer of pointer
  - **Structure : a kind of cluster containing different modalities**
    - Very useful to handle data of different modalities in a unified way
    - Key technique in considering the data structure